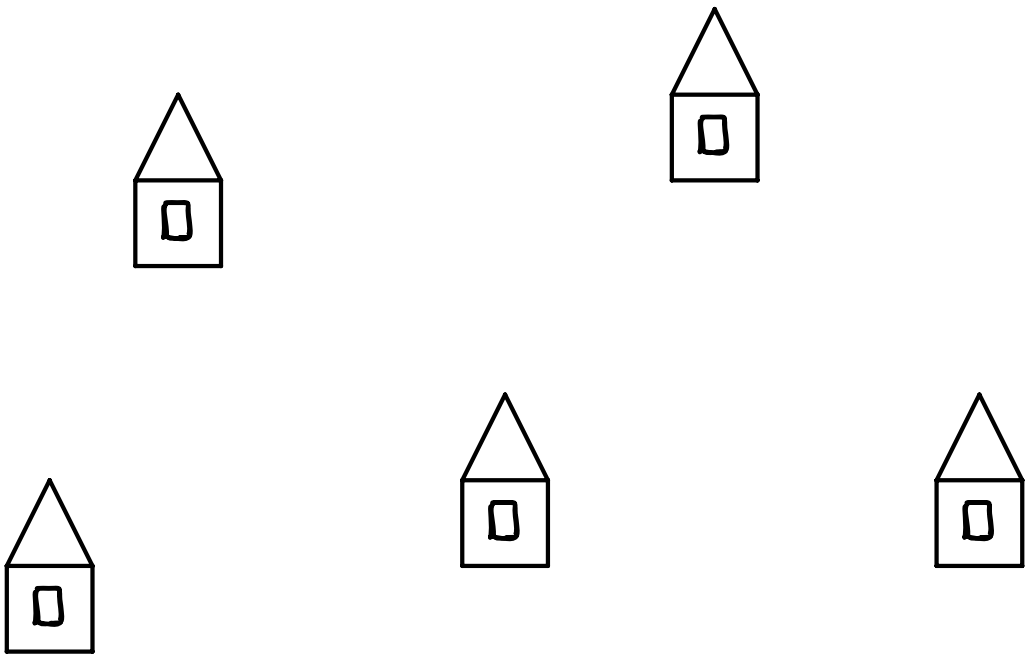
Minimum Spanning Trees (MST)

[Ref: CLRS Chapter 23]

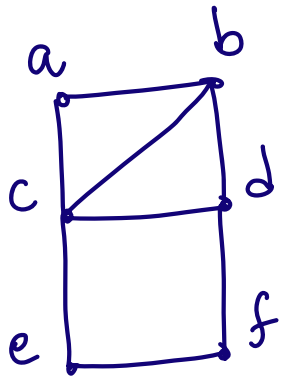
Network Design:



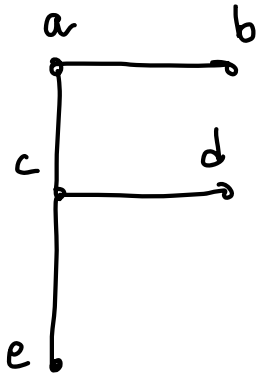
As an internet service provider you want to connect the houses with a minimum total cost.

Q: How to model this problem using graphs

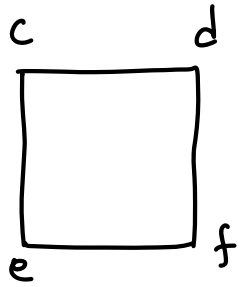
Recap: A **Subgraph** H of a graph G is a graph whose set of vertices and set of edges are all subsets of G.



Graph G



H_1

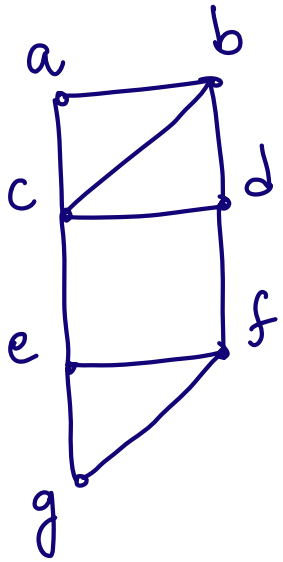


H_2

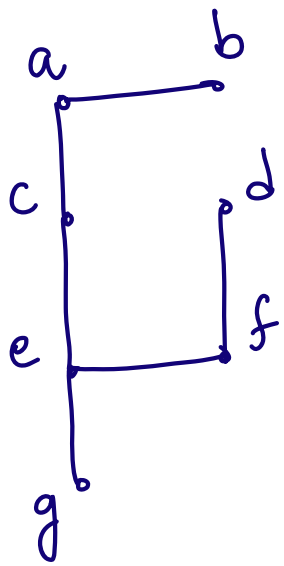
Both H_1 and H_2 are subgraphs of G

- **Spanning tree** T of an undirected graph G is a subgraph that is a tree which includes all the vertices of G.

E.g:



Graph G



Spanning tree of G

TRUE | FALSE :

Every Connected graph Contains a Spanning tree

For the rest of this topic,

we assume that graphs are

- Undirected
- Connected
- Edge weighted

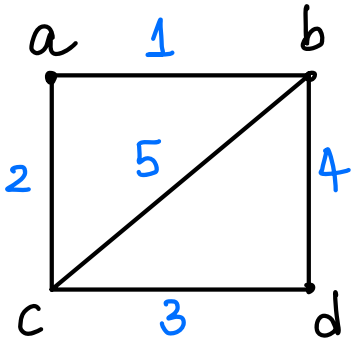
Minimum Spanning Tree (MST)

The minimum Spanning tree Problem is given a graph $G=(V, E)$ together with a function $w: E \rightarrow \mathbb{R}$ that assigns a real weight w_e to each edge e, find a spanning tree T of minimum weight.

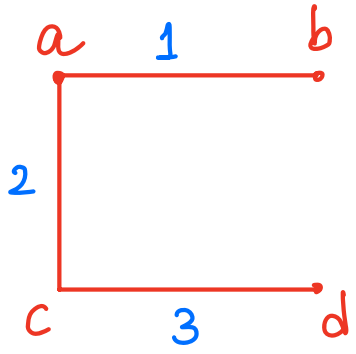
Where the weight of a tree T is defined as

$$w(T) = \sum_{e \in T} w_e$$

Example:



Graph G

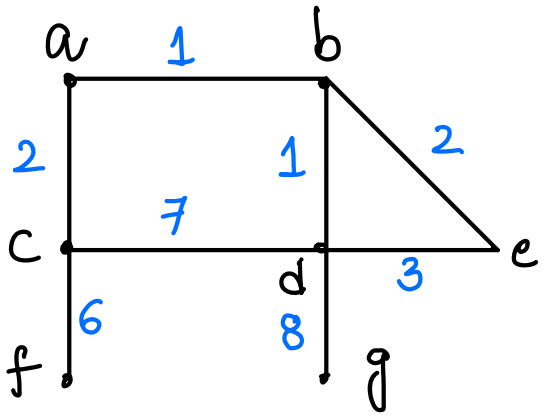


MST T

Weight of $T = w(T) = 1 + 2 + 3$

$= 6$

Example 2:



Graph G

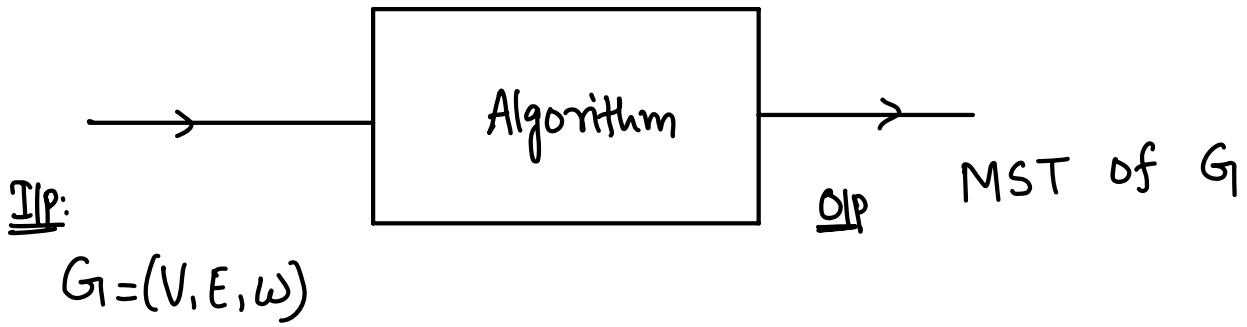
Q: What is the minimum Spanning tree of G?

What is its weight

MST Problem

Input: An Edge weighted Connected Undirected graph G

Output: A Minimum Spanning tree of G



TRUE | FALSE:

There can be more than one MST for a graph.

Note: If every edge in G_1 has weight 1 then every spanning tree of G_1 is a MST with weight $n-1$.

We look at two algorithms for MST Problem

- Kruskal's algorithm
- Prim's algorithm

} Both are greedy Algorithms.

Main Idea

Greedy strategy is captured by the following generic method, which grows the MST one edge at a time.

We maintain a set of edges A, such that

"Prior to each iteration, A is a subset of some MST." —(I)

At each step, we determine an edge (u,v) that we can add to A without violating above property (I), in the sense that A ∪ {(u,v)} is also subset of a MST. We call such an edge a **Safe edge** for A.

GENERIC-MST(G, w)

```
1  A = ∅
2  while A does not form a spanning tree
3      find an edge (u, v) that is safe for A
4      A = A ∪ {(u, v)}
5  return A
```

The tricky Part :

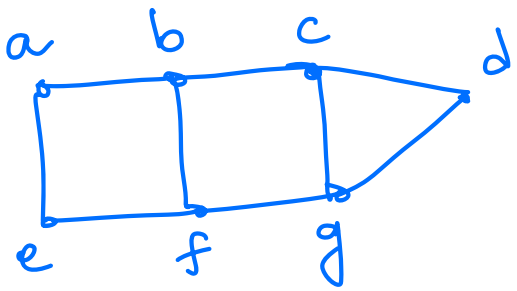
How to find a safe-edge in line 3 ?

In the remainder of this class, we learn a method for selecting safe edges.

Some definitions

- A **cut** $(S, V-S)$ of an undirected graph $G=(V,E)$ is a partition of V.

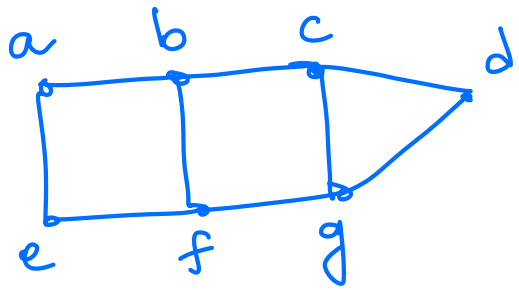
Ex:



$S = \{a, b, e, f\}$, $V-S = \{c, g, d\}$

$(S, V-S)$ is a cut.

- We say that an edge $uv \in E$ **Crosses** the cut $(S, V-S)$ if one of its endpoints is in S and the other is in $V-S$.

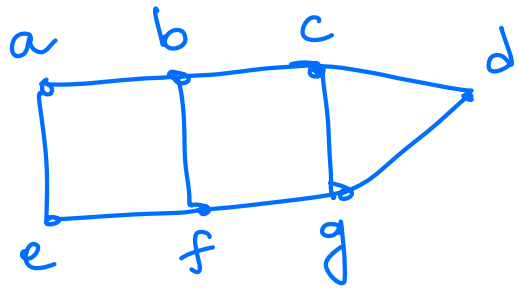


$S = \{a, b, e, f\}$, $V-S = \{c, g, d\}$

The edge bc crosses the cut

" " fg " " "

- We say that a cut respects a set A of edges if no edge in A crosses the cut.



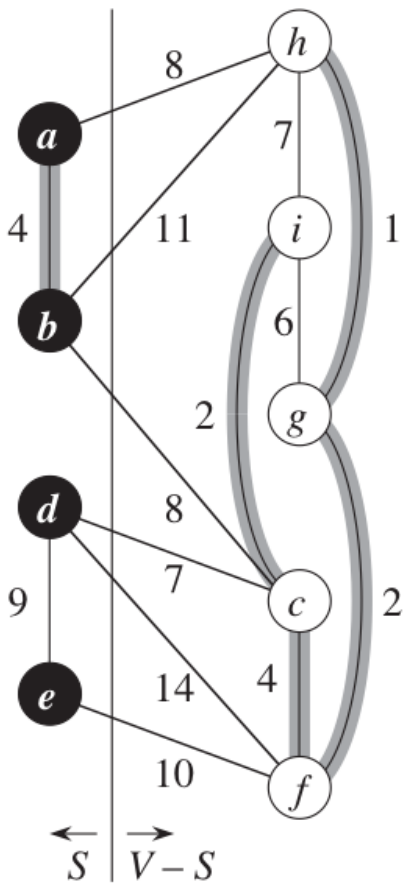
$S = \{a, b, e, f\}$, $V - S = \{c, g, d\}$

$A = \{ae, ef\}$

Here the cut $(S, V-S)$ respects A.

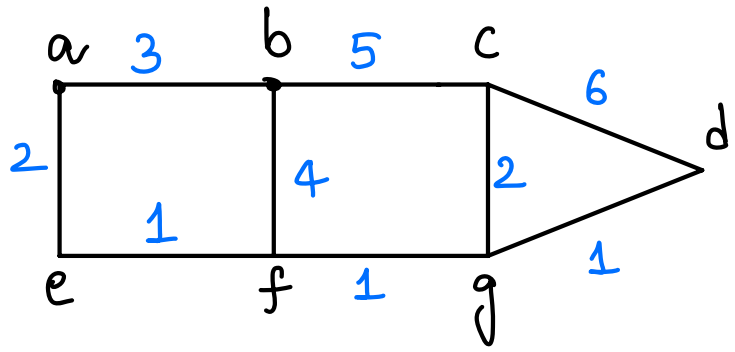
- An edge is a **light edge** crossing a cut if its weight is the minimum of any edge crossing the cut.

Example:



The edge dc is the unique light edge crossing the cut.

Warmup Questions [based on above definitions]



$S = \{a, b, e, f\}$, $V-S = \{c, g, d\}$

$A = \{ae, ef\}$

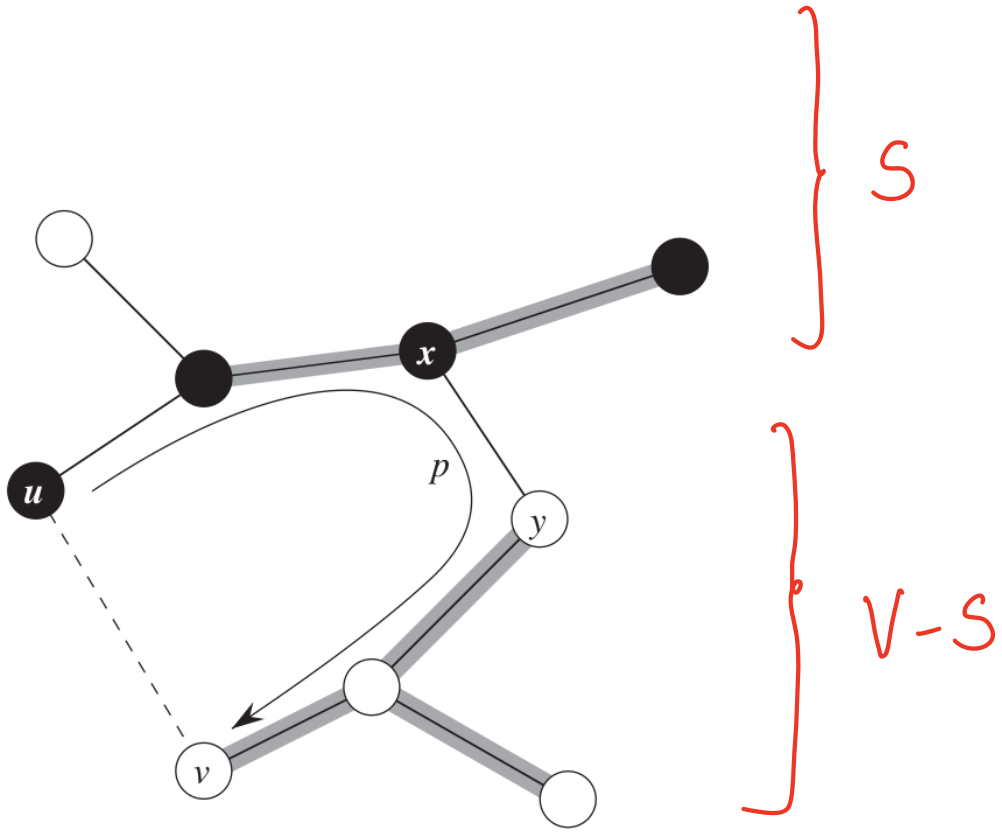
Q1: Does the cut $(S, V-S)$ respects A ?

Q2: Which edges are crossing the cut ?

Q3: What is the light edge crossing the cut ?

Theorem: [Ref: Cormen et al] **Theorem 23.1**

Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E. Let A be a subset of E that is included in some minimum spanning tree for G, let $(S, V - S)$ be any cut of G that respects A, and let (u, v) be a light edge crossing $(S, V - S)$. Then, edge (u, v) is safe for A.


x
u
p
y
v
S
V-S

Back to the Algorithm

GENERIC-MST(G, w)

```
1  A = ∅
2  while A does not form a spanning tree
3      find an edge (u, v) that is safe for A
4      A = A ∪ {(u, v)}
5  return A
```

TRUE | FALSE:

At any point (step) in the execution of the above algorithm, the graph $G_A = (V, A)$ is a forest.

[ie, each component of G_A is a tree]

TRUE | FALSE :

Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E. Let A be a subset of E that is included in some minimum spanning tree for G, and let $C = (V_C, E_C)$ be a connected component (tree) in the forest $G_A = (V, A)$. If (u, v) is a light edge connecting C to some other component in G_A, then (u, v) is safe for A.

Warmup - Questions

TRUE | FALSE:

Let G be a graph with distinct edge weights.

(a) The minimum weight edge is always be a Part of MST.

(b) The Maximum weight edge is never be a Part of any MST.

Algorithms to find MST

The two MST algorithms we study, use a specific rule to determine a safe edge in our GENERIC-MST.

Kruskal's Algorithm [outline]

The algorithm maintains a set of edges A that eventually forms an MST.

At any intermediate step $G_A = (V, A)$ is a forest

Initial step:

The algorithm starts by treating every vertex in the graph as a separate tree. So begin with $|V|$ trees

Sorting

All edges in the graph are sorted in non-decreasing order of their weight :

$$w(e_1) \leq w(e_2) \text{ --- } \leq w(e_m)$$

Safe edge Selection :

The algorithm iterates through the edges in Sorted order.

For each uv, it checks

Do u & v belong to different trees ?

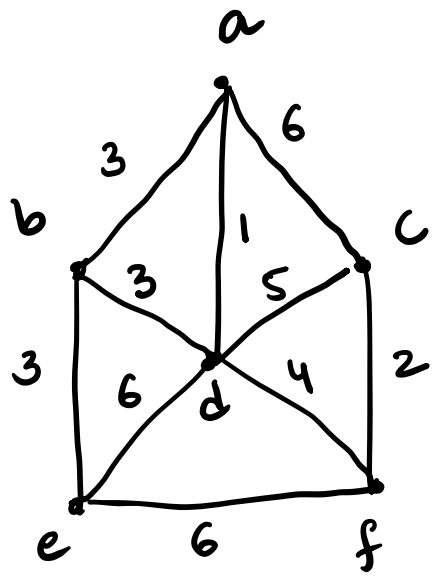
if YES, The edge is added to A
[It merges two trees]

if NO, adding the edge would create a cycle

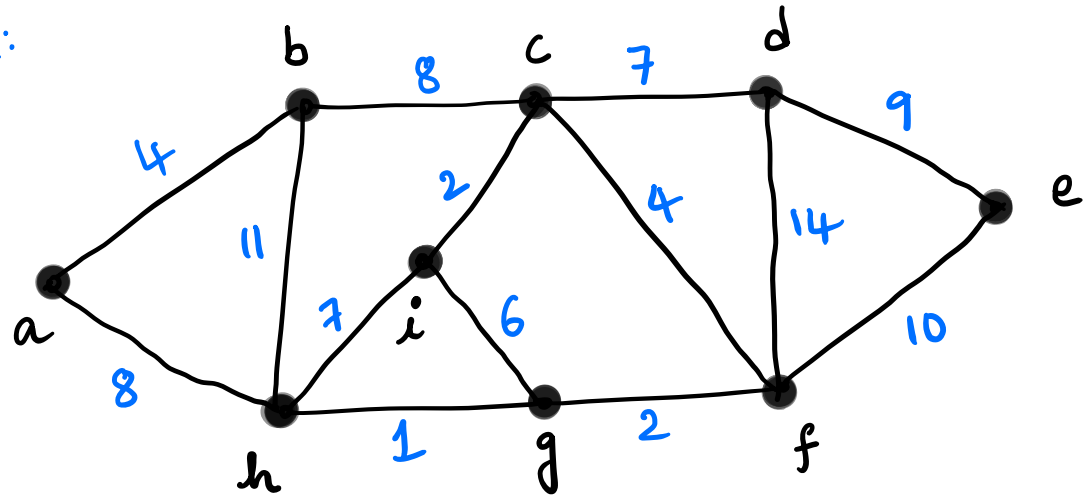
$\therefore$ The edge is discarded

```
KRUSKAL-NAIVE(G)
1   A = ϕ
2   Sort edges E in increasing order of weight
3   for each edge uv in sorted order
4       if u and v are in different trees in G_A
5           A = A ∪ {uv}
6   return A
```

Ex 1



Ex2:



Q: Find the minimum spanning tree of G using Kruskal's algorithm. What is its weight?

Running time

KRUSKAL-NAIVE(G)

1 $A = \phi$

2 Sort edges E in increasing order of weight } Sorting

3 for each edge uv in sorted order

4 if u and v are NOT connected in $(V, A) = G_A$

5 $A = A \cup \{uv\}$

6 return A

Checked using BFS(u) or DFS(u) in G_A

Running time: $O(E(V+E))$

Better Algorithm Using UNION-FIND

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3      MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6      if FIND-SET(u) ≠ FIND-SET(v)
7          A = A ∪ {(u, v)}
8          UNION(u, v)
9  return A
```

Creates a set whose only member is v.

returns the representative element of the set containing v.

Unites the sets that contain u & v.

line 1 - $O(1)$

lines 2&3 - $O(V)$

line 4 - $O(E \log E)$

lines 5 to 8 - line 5 runs $O(E)$ times

We can check FIND-SET(u) ≠ FIND-SET(v) using either BFS or DFS (But this not optimal)

UNION(u, v): merging two connected components

$O(V+E)$

Running time: $O(E(V+E))$

↓ can be improved to

$O(E \log V)$

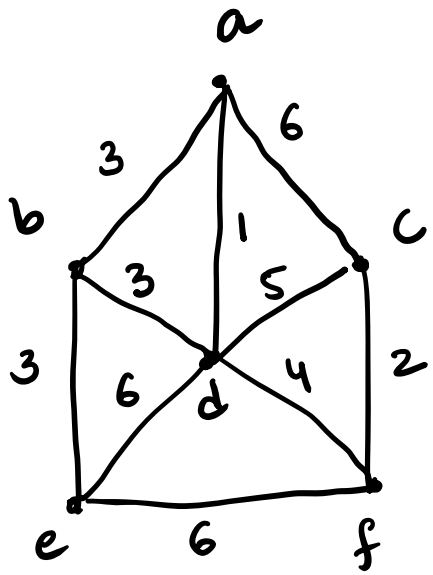
Prim's algorithm

- The Set A always forms a single tree.

  [Initially tree will contain one vertex without any edges]

- The safe edge added to A is the least-weight edge connecting the existing tree to a vertex not yet included in the tree

Example :



Implementing Prim's algorithm

r = root vertex

$u.key$ = minimum weight of any edge connecting v to a vertex in the tree.

$u.\pi$ = Parent of v in the tree.

Q = min Priority queue

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

The MST A for G is

$$A = \{ (v, v.\pi) : v \in V - \{r\} \}$$

Running time

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

O(V)

While loop runs |V| times

Each EXTRACT-MIN operation $O(\log |V|)$ time

Total time spent — $O(|V| \log |V|)$

For loop lines 8 to 11 executes $O(|E|)$ times altogether

because Sum of the degrees $= 2|E|$.

line 9 : $v \in Q$ can be checked in constant time by keeping a bit for each vertex, that tells whether or not it is in Q.

line 11 : involves an implicit DECREASE-KEY operation on the min-heap, which takes $O(\log|V|)$ time.

Total time for Prim's algorithm is

$$O(|V| \log|V| + |E| \log|V|)$$

$$= O(|E| \log|V|)$$



can be improved using Fibonacci heap

$$O(|E| + |V| \log|V|)$$

Boruvka's algorithm [Ref: Chap 7.3 in Jeff Erickson book]

Idea is to add all the safe edges and recurse.

That is, In every iteration,

Every existing component C identifies its safe (Cheapest) edge connecting C to a outside Vertex.

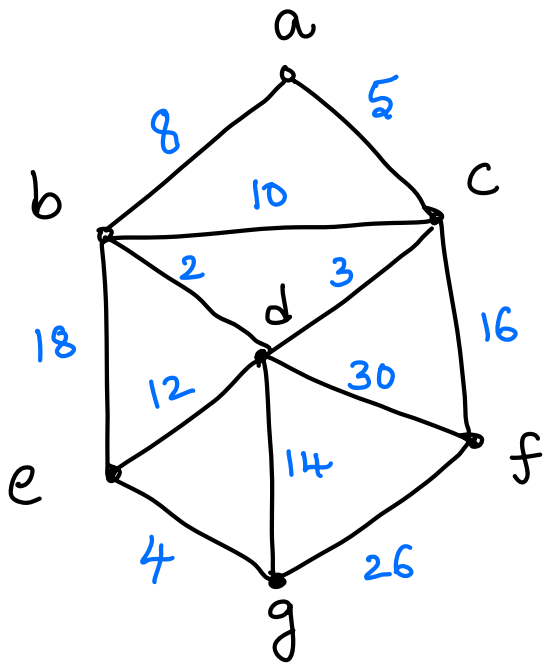
BORŮVKA (G, ω)

```
1  Intialize F as a forest where each vertex is
     its own tree

2  While F has more than one connected component
3        For each component C in F
4              find the cheapest edge uv such that
                   u is inside C, v is outside C
5        add all such unique cheapest edges to F
6  Return F
```

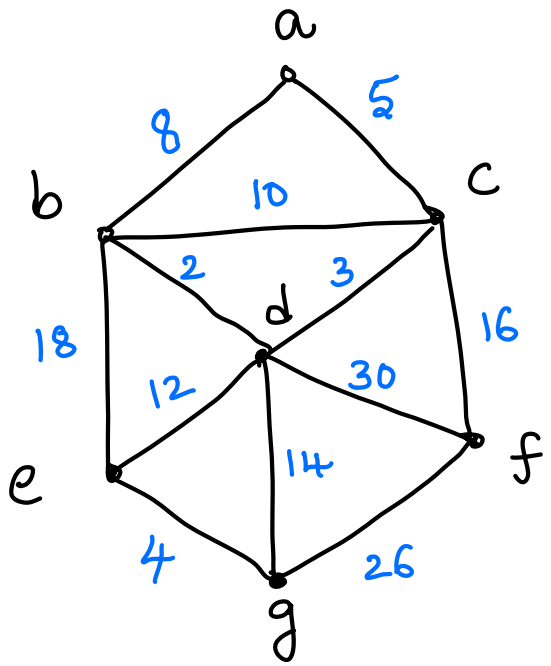
Example:

We apply Boruvka's algorithm on the following example.

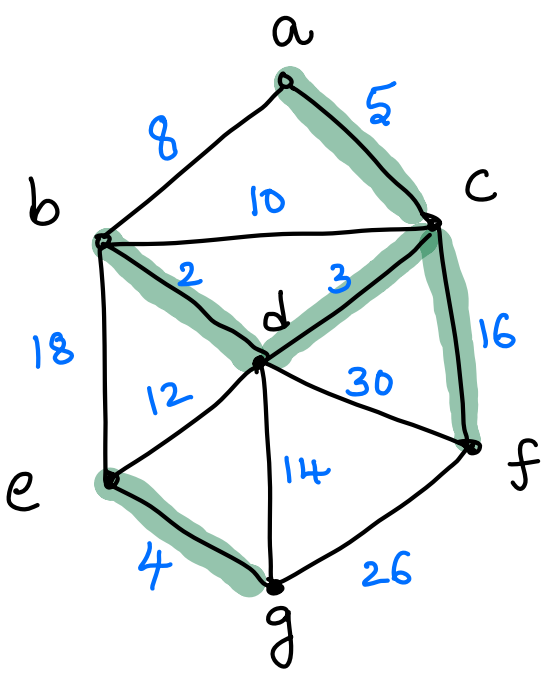


Example:

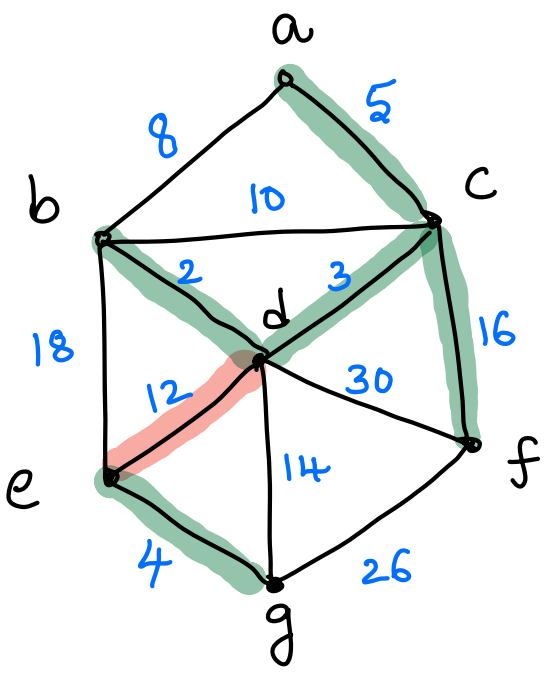
We apply Boruvka's algorithm on the following example.



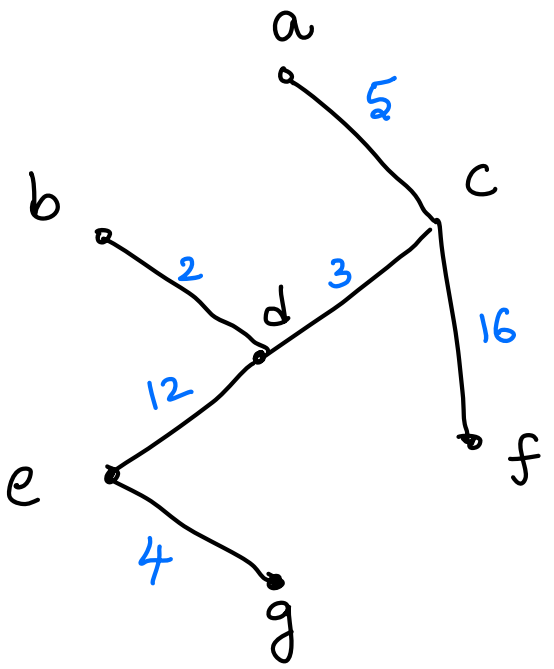
1st step: All green edges are added.



2nd step only edge de is added.



Final MST Obtained



Running time: Every step the number of components is reduced by at least half.

$\therefore$ So total # iterations $O(\log |V|)$

Each iteration looks at every edge once.

total running time $O(|E| \log |V|)$

Tie-breaking rule in Borůvka's Algorithm :

For example, consider a triangle graph with nodes {a, b, c} all edges of weight 1. Then a cycle could be created if we select ab as minimal weight edge for {a}

bc as minimal weight edge for {b}

ca as minimal weight edge for {c}

A tie breaking rule which orders edges first by source, then by destination will prevent creation of a cycle resulting in the minimal spanning tree {ab, bc}

Strange Spanning tree :- (SST)

A SST of a graph G is a Spanning tree of G whose largest edge weight is minimum over all Spanning trees of G.

The weight of the SST is the weight of the maximum weight edge in T.

TRUE/False:

① Any MST is a SST

② Any SST is a MST

Exercise Problems

① Suppose we are given both an undirected graph G with weighted edges and a MST T of G.

ⓐ Describe an algorithm to update the MST when the weight of a single edge e is decreased.

ⓑ Describe an algorithm to update the MST when the weight of a single edge e is increased.

In both the cases, the input to your algorithm is the edge e and its new weight.

Your algorithm should modify T so that it is still a MST.

② The second smallest spanning tree of a given graph is the spanning tree of G with smallest total weight except for the MST.

Describe an algorithm to find the second smallest spanning tree of a given graph G.